Ders 4 Genişletilmiş Notu: Benzer Dokümanlar: MinHashing ve Locality Sensitive Hashing

Ders 4 Genişletilmiş Notu: Benzer Dokümanlar: MinHashing ve Locality Sensitive Hashing

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Problemin Tanımı ve Büyüklüğü
- 2. Benzer Doküman Bulmanın Kullanım Alanları
- 3. Üç Aşamalı Süreç
- 4. Shingling ve Jaccard Benzerliği (Tekrar)
- 5. Dokümanları 0-1 Vektörlerine Çevirmek: Girdi Matrisi
- 6. MinHashing ve İmza Matrisi
- 7. Hash Fonksiyonlarıyla MinHashing (Permütasyonsuz)
- 8. Locality Sensitive Hashing (LSH)
- 9. LSH'de Hata Payı: Sayısal Örnek
- 10. Kapanış: Örnek Sorular ve Sonraki Ders
- Örnek Sorular
- Dersin Toparlaması

1. Problemin Tanımı ve Büyüklüğü

Hoca derse, 3. bölümde (chapter 3) şu ana kadar gelinen yerin hızlı bir tekrarıyla başladı ve dersin Türkçe ile İngilizce arasında gidip geleceğini söyledi. Bölümün konusu büyük veri kümelerinde benzer kümeleri bulmaktır (Finding Similar Sets in Large-Scale Datasets). Elimizde büyük bir doküman topluluğu (document corpus) vardır ve hangi dokümanların birbirine benzediğini bulmak isteriz.

Bunun bir büyük veri problemi olmasının sebebi iki yönlüdür: doküman sayısı çok fazladır ve her dokümanın içeriği de çok büyüktür. Veri tek bir bilgisayarın belleğine ya da diskine sığmaz. Bu yüzden veriyi birden fazla dağıtık bilgisayara yaymak ve sonra analiz etmenin bir yolunu bulmak gerekir. Hoca bu bölümde öğrenilecek algoritmaların problemi tek bir bilgisayarın belleğiyle bile çözmeye yardım ettiğini vurguladı.

2. Benzer Doküman Bulmanın Kullanım Alanları

Hoca geçen hafta gösterilen örnekleri tekrar etti:

- **Benzer dokümanların tespiti ve sınıflandırılması:** En önemli uygulama budur. Birbirine benzeyen dokümanlar belirli bir kategori altında toplanabilir.
- **Film önerisi (Netflix gibi):** Çok kullanıcılı bir yayın (streaming) servisinde her filmin izleyicileri listelenir. Aynı kişiler tarafından izlenen filmler aynı film zevkine sahip insanlara hitap ediyordur, bu yüzden aynı kategoride düşünülebilir. Bir kullanıcı bu kategorideki bir filmi izlemişse, aynı sınıftaki başka bir film ona önerilebilir. Hoca bunun ters yönde de (kullanıcılar üzerinden) yapılabileceğini söyledi.

- **Sosyal medya hesaplarını eşleştirme:** Bir kişinin LinkedIn, Facebook, Instagram gibi farklı platformlarda farklı kullanıcı adlarıyla profilleri olabilir. Herkese açık özelliklerden yola çıkarak birbirine benzeyen profiller aynı kategoride toplanır. Böylece kullanıcı adları farklı olsa bile "bu Instagram, bu Facebook ve bu LinkedIn hesabı muhtemelen aynı kişiye aittir" denebilir.
- **Arama motorları:** Aynı içeriği gösteren birden fazla bağlantıyı listelemek istemezler. İçerik olarak benzer sayfaları tespit edip yalnız birini gösterirler.
- **İntihal (plagiarism) tespiti:** Bir çalışmanın gerçekten yazarına ait olup olmadığı, izinsiz alıntı içerip içermediği kontrol edilirken çalışmanın içeriği internetteki mevcut dokümanlarla karşılaştırılır. Bunun için de bu bölümdeki algoritmalar kullanılır.
- **Haber siteleri (Google News gibi):** Farklı kaynaklardan gelen haberleri listelerler. Birden fazla kaynakta yayınlanmış aynı haberi sonuçlarında tekrar tekrar göstermemek için benzer dokümanları tespit ederler.

3. Üç Aşamalı Süreç

Geçen hafta bu uygulamaları gerçekleştirmek için üç aşamalı bir süreçten bahsedilmişti:

1. Shingling,
2. MinHashing,
3. Locality Sensitive Hashing (LSH). Bu derste tamamlanacak.

Hedef şöyle tarif edildi: Elimizde, örneğin, 1 milyon doküman var ve her biri 3 GB büyüklüğünde olsun. Elle yapacak olsak bütün olası ikililerin, yani 1 milyonun ikili kombinasyonu kadar çiftin benzerliğini hesaplar, belirli bir eşiğin (örneğin %80) üzerinde kalanları ayırırdık. Mesela 1 milyon doküman içinde 80 dokümanın bu benzerlik oranının üzerinde kaldığını bulurduk. Fakat bütün ikililere bakmak, üstelik dokümanlar gigabayt mertebesindeyken, inanılmaz zaman alır. Bu kadar dokümanı tek bir belleğe yerleştirmek de imkânsızdır. Bu yüzden dokümanların benzerliği yakalayabilecek şekilde onları temsil eden imzaları (signature) oluşturulur.

4. Shingling ve Jaccard Benzerliği (Tekrar)

4.1 Shingling

Doküman olduğu gibi kullanılmaz. İçinden dokümanı anlatan özellikler çıkarılır ve doküman bir kümeye dönüştürülür. İlk aşama shingling'dir. Girdisi bir dokümandır. Parametre olarak hangi uzunlukta karakter dizileri çıkarmak istediğimizi (k) veririz. Algoritma bu uzunluktaki karakter dizilerini çıkarıp bir küme olarak döndürür.

Hocanın oyuncak örneği: Doküman beş karakterden oluşan `abcab`, parametre $k = 2$ olsun. 2 uzunluğundaki shingle'lar `ab`, `bc`, `ca` olur. Dokümanda `ab` iki kez geçer, ama sonuç bir küme olduğu için aynı eleman ikinci kez yazılmaz.

Bu fikir doğal dil işlemede (Natural Language Processing) n-gram olarak da geçer: Verilen bir dokümanı ilk karakterinden başlayarak karakter dizilerine (karakter katarlarına) dönüştürmek. Doküman bu kümeye dönüştürüldükten sonra artık bu kümeyle ifade edilir. İki doküman arasındaki benzerliğe bakmak, iki küme arasındaki benzerliğe bakmak olur.

4.2 Jaccard benzerliği

İki küme arasındaki benzerlik için yaygın olarak Jaccard benzerliği (Jaccard similarity) kullanılır. Kesişimin eleman sayısı paya, birleşimin eleman sayısı paydaya konur:

$$\text{sim}(C_1, C_2) = \frac{|C_1 \cap C_2|}{|C_1 \cup C_2|}$$

Geçen haftaki örnekte iki kümenin kesişimi 3, birleşimi 8 elemanlıydı. Dolayısıyla benzerlik $3/8 = 0.375$'tir.

4.3 İsteğe bağlı sıkıştırma: Shingle'ları tamsayıya çevirmek

Shingle'lar (k = 2, 3, 5 gibi) bellekte karakter dizisi (string) olarak tutulmak istenmeyebilir. Bir hash fonksiyonu yazılarak her karakter dizisi daha az yer tutan bir tamsayıya (integer) dönüştürülebilir. Bir tamsayı 4 bayttır. 1 milyon farklı karakter dizisi varsa bellekte $10^6 \times 4$ bayt yer tutulur. Hash fonksiyonu karakter dizilerini 4 baytlık tamsayı uzayının değerlerine eşler (map).

Hash fonksiyonunun çözünürlüğü iyi olmadığında iki farklı karakter dizisi aynı tamsayıya düşebilir. Böyle durumlar azsa, kazanılan bellek daha önemli kabul edilir. Hoca bunun isteğe bağlı (opsiyonel) bir sıkıştırma olduğunu, özellikle belleğin sınırlı olduğu durumlarda tercih edilebileceğini söyledi.

5. Dokümanları 0-1 Vektörlerine Çevirmek: Girdi Matrisi

Dokümanları bellekte alt alta yazılmış karakter dizileri olarak tutmak yerine vektör olarak tanımlarız. Hocanın örneği: 1 milyon doküman var ve bu dokümanların hepsinde toplam 700 farklı k uzunluğunda shingle var. Bu 700 shingle birinciden yedi yüzüncüye kadar yukarıdan aşağı yazılır ve bir "karakter dizisi uzayı" oluşur. Sonra her doküman için şuna bakılır: Birinci shingle bu dokümanda varsa 1, yoksa 0 yazılır. Aynı şey 700. shingle'a kadar yapılır.

Böylece her doküman 0 ve 1'lerden oluşan bir ikili (binary) vektör olur. 1 milyon doküman varsa 1 milyon vektör oluşur. Her vektörün boyutu shingle uzayının boyutu kadardır, bu örnekte 700.

Bu vektörler yan yana konunca bir girdi matrisi (input matrix) elde edilir:

- Kolonlar dokümanlardır.
- Her satır bir k uzunluğundaki shingle'a karşılık gelir.
- Bir dokümanın kolonunda bir satırda 0 varsa o shingle dokümanda yoktur, 1 varsa vardır.

5.1 Jaccard benzerliğini matris üzerinden hesaplamak

Bu gösterimin faydası, Jaccard benzerliğinin çok kolay hesaplanmasıdır. İki dokümanın kesişimi, ikisinde de 1 olan satırlardır. Birleşim, ikisinden en az birinde 1 olan satırlardır. Slayttaki örnekte ikisinde de 1 olan satır sayısı 2, en az birinde 1 olan satır sayısı 5'tir. Benzerlik $2/5 = 0.4$ olur.

Hoca, kodunu yazacağımız için bunu formülleştirdi. İki kolon karşılaştırılırken bir satır dört tipten biridir:

Tip	C_1	C_2
a	1	1
b	1	0
c	0	1
d	0	0

Jaccard benzerliğinin payında yalnız a tipi satırlar, paydasında a, b ve c tipi satırlar vardır. d tipi satırlar hesaba girmez:

$$\text{sim}(C_1, C_2) = \frac{a}{a + b + c}$$

Buraya kadar olan kısım shingling süreciydi: Dokümanlar kümelere, kümeler 0-1 kolon vektörlerine dönüştürüldü ve bir matris oluşturuldu. Amaç, çok sayıda doküman içinde birbirine benzeyenleri bulmaktır.

6. MinHashing ve İmza Matrisi

6.1 Neden imza matrisi?

Dokümanların sayısı çok fazla, her dokümanın satır sayısı da çok büyük olabilir. Böyle bir girdi matrisi tek bir belleğe sığmaz. Bu yüzden girdi matrisini daha küçük boyutlu bir matrisle temsil etmek isteriz. Ama bu matris öyle olmalı ki iki doküman arasındaki benzerlik bu küçük matris üzerinden de bulunabilsin. Bu temsilci matrise imza matrisi (signature matrix), onu üreten algoritmaya MinHashing denir.

6.2 İmza matrisi nasıl oluşur?

MinHashing şunu söyler: İmza matrisinin her satırı için bir hash fonksiyonu oluşturulur. Hash fonksiyonu girdi matrisindeki bir dokümanın kolon vektörünü alır ve karşılığında bir sayısal değer üretir. Bu değer imza matrisinde o hash fonksiyonunun satırına, o dokümanın kolonuna yazılır. Bu işlem her doküman ve her hash fonksiyonu için yapılır. Slayttaki örnekte 4 doküman ve 3 hash fonksiyonu vardı. Sonuç 3 satır, 4 kolonluk bir imza matrisidir.

6.3 İmza matrisinde benzerlik

İmza matrisinde iki dokümanın benzerliği için Jaccard benzerliği kullanılmaz. Bunun yerine şu oran kullanılır:

$$\text{sim}_{\text{imza}}(C_i, C_j) = \frac{\text{değeri aynı olan satır sayısı}}{\text{toplam satır sayısı}}$$

Hoca slayttaki imza matrisi üzerinde üç ikiliyi hesapladı (önce C_1-C_2 ile başlayıp vazgeçti ve C_1-C_3'e geçti):

- C_1 **ve** C_3**:** İki satırda değerler aynı (hocanın okumasıyla "2'ye 2" ve "1'e 1"), bir satırda farklı. Benzerlik $2/3$.
- C_2 **ve** C_4**:** Birinci, ikinci ve üçüncü satır aynı. Benzerlik $3/3 = 1$, yani %100 benzer.
- C_1 **ve** C_2**:** Üç satırın üçü de farklı. Benzerlik $0/3 = 0$.

6.4 MinHashing'in temel ilkesi

Yeterince fazla hash fonksiyonu kullanıldığında, iki dokümanın imza matrisindeki benzerliği, aynı iki dokümanın girdi matrisindeki Jaccard benzerliğine çok yakın çıkar, ona yakınsar. Hoca bunu algoritmanın en önemli prensibi olarak verdi. Slayttaki İngilizce ifade şuydu: İki imzanın beklenen benzerliği, imzaların temsil ettiği kolonların (kümelerin) Jaccard benzerliğine eşittir.

Bir adım geri atarsak: Girdi matrisinde her dokümandaki shingle sayısı çok fazlaysa Jaccard benzerliğini bulmak için harcanan zaman çok fazla olur. İmza matrisinde satır sayısı çok daha azdır ve oradaki benzerlik Jaccard benzerliğine yakınsar. Bu yüzden benzer dokümanlar imza matrisi üzerinden tespit edilebilir.

> **Ek bilgi (derste anlatılmadı):** Bu ilkenin gerekçesi büyük olasılıkla kitapta permütasyon tabanlı MinHash için verilen bir olasılık sonucudur. Satırlar rastgele permüte edildiğinde, iki kolonda da ilk karşılaşılan "en az birinde 1 olan" satır, a tipi (ikisinde de 1) satırlardan biri olma olasılığıyla aynı MinHash değerini üretir. Bu olasılık $a/(a+b+c)$, yani tam olarak Jaccard benzerliğidir. Çok sayıda bağımsız hash fonksiyonunda aynı çıkan satırların oranı bu olasılığın bir tahminidir.

6.5 Permütasyon tabanlı hash fonksiyonu

Geçen hafta konuşulan hash fonksiyonu türünde hash fonksiyonu, satırların karışık bir sırasıdır (permuted order). Slayttaki örnekte 7 satır vardı. Bir doküman için hash değeri şöyle bulunur: Permütasyon sırasıyla satırlar gezilir. İlk 1 görülen satırın numarası o hash fonksiyonunun o doküman için değeridir.

- **1. doküman için:** Permütasyon sırasında 1. satırda 0, 2. satırda 1 görüldü. İlk 1'e karşılık gelen değer C_1'in bu hash fonksiyonu için hücresine yazıldı.
- **2. hash fonksiyonu, 3. doküman için:** Permütasyon sırasında 1., 2. ve 3. satırlarda 0, 4. satırda 1 var. İlk 1 görülen sıra 4 olduğu için 2. hash fonksiyonunun 3. doküman hücresine 4 yazıldı.

6.6 Permütasyonun bellek sorunu

Permütasyon büyük veride uygun bir yöntem değildir. Girdi matrisinde 1 milyon satır olsaydı, her hash fonksiyonu için bellekte 1 milyon tamsayılık bir permütasyon tutmak gerekirdi. Her eleman için bir tamsayı (4 bayt) düşünülürse, yalnız bir hash fonksiyonunun permütasyonu için $10^6 \times 4$ bayt gerekir. Bu yüzden hash fonksiyonu iyi olmasına rağmen çok bellek tuttuğu için tercih edilmez. Bunun yerine başka hash fonksiyonları ve bunlarla birlikte izlenen bir algoritma kullanılır.

7. Hash Fonksiyonlarıyla MinHashing (Permütasyonsuz)

Hoca algoritmayı önce bir örnek üzerinden, sonra sözde kod (pseudocode) üzerinden anlattı.

7.1 Örneğin kurulumu

- Girdi matrisinde 5 satır ve 2 doküman (C_1, C_2) var. Satırlar 1'den 5'e numaralı.
- İki hash fonksiyonu seçilmiş:
  - $h_1(x) = x \bmod 5$
  - $h_2(x) = (2x + 1) \bmod 5$
- 2 hash fonksiyonu ve 2 doküman olduğu için imza matrisi 2×2 olur.

Hocanın satır satır anlatımından girdi matrisi şöyle kurulur (ara adım eklendi: tablo, hocanın her satırda söylediği 0/1 değerlerinden derlendi):

Satır	C_1	C_2
1	1	0
2	0	1
3	1	1
4	1	0
5	0	1

7.2 Algoritmanın kuralları

Algoritma yinelemeli (iterative) çalışır: 1. satırdan 5. satıra kadar bütün satırlar için tekrarlanır.

- **Başlangıç:** İmza matrisinin bütün hücrelerine sonsuz (∞) yazılır.
- **Hash değerleri:** Her satırda önce satır numarası hash fonksiyonlarına verilir ve h_1, h_2 değerleri hesaplanır.
- **Güncelleme:** Her doküman için o satırdaki girdi değerine bakılır:
  - 0 ise o dokümanın imza hücrelerine dokunulmaz. Sonsuzsa sonsuz kalır.
  - 1 ise her hash fonksiyonu için, hash değeri hücredeki mevcut değerden küçükse hücre hash değeriyle güncellenir. Yani hücreye ikisinden küçüğü yazılır.

7.3 Satır satır çözüm

1. iterasyon (satır 1): $h_1(1) = 1 \bmod 5 = 1$. $h_2(1) = (2 \cdot 1 + 1) \bmod 5 = 3 \bmod 5 = 3$. İmza matrisinin dört hücresinde de ∞ var. C_1'de bu satırda 1 var, güncellenir: $\min(\infty, 1) = 1$, $\min(\infty, 3) = 3$. C_1'in imzası $(1, 3)$. C_2'de 0 var, güncelleme yok, (∞, ∞) kalır.

2. iterasyon (satır 2): $h_1(2) = 2$. $h_2(2) = (4 + 1) \bmod 5 = 5 \bmod 5 = 0$. C_1'de 0 var, önceki iterasyondan kalan $(1, 3)$ aynen kalır. C_2'de 1 var: $\min(\infty, 2) = 2$, $\min(\infty, 0) = 0$. C_2'nin imzası $(2, 0)$.

3. iterasyon (satır 3): $h_1(3) = 3$. $h_2(3) = (6+1) \bmod 5 = 7 \bmod 5 = 2$. C_1'de 1 var: $\min(1,3) = 1$ (aynen kalır), $\min(3,2) = 2$ (güncellenir). C_1'in imzası $(1,2)$. C_2'de de 1 var: $\min(2,3) = 2$, $\min(0,2) = 0$. Mevcut değerler küçük olduğu için güncelleme olmaz, $(2,0)$ kalır.

4. iterasyon (satır 4): $h_1(4) = 4$. $h_2(4) = 9 \bmod 5 = 4$ (ara adım eklendi; hoca yalnız "4'e 4" dedi). C_1'de 1 var: önceki $(1,2)$, hash değerleri $(4,4)$. Önceki değerler küçük, $(1,2)$ kalır. C_2'de 0 var, güncelleme yok.

5. iterasyon (satır 5): $h_1(5) = 0$. $h_2(5) = 11 \bmod 5 = 1$ (ara adım eklendi; hoca "0 ve 1 çıkmış" dedi). C_1'de 0 var, güncelleme yok. C_2'de 1 var: $\min(2,0) = 0$, $\min(0,1) = 0$. C_2'nin imzası $(0,0)$.

Özet tablo:

Satır	C_1	C_2	h_1	h_2	C_1 imzası	C_2 imzası
1	1	0	1	3	(1, 3)	(∞, ∞)
2	0	1	2	0	(1, 3)	(2, 0)
3	1	1	3	2	(1, 2)	(2, 0)
4	1	0	4	4	(1, 2)	(2, 0)
5	0	1	0	1	(1, 2)	(0, 0)

Satır sayısı kadar (5) iterasyondan sonra elde kalan matris imza matrisidir: C_1'in imzası $(1,2)$, C_2'nin imzası $(0,0)$. Bu sonuç python ile ayrıca doğrulandı.

Bu yöntemin permütasyona göre avantajı, permütasyonları fiziksel bellekte tutmaya gerek kalmamasıdır.

7.4 Sözde kod

Hoca örneği sözde kodla eşleştirdi:

- En dışta bütün satırlar için çalışan bir "for each" döngüsü vardır. Hoca "en önemlisi buna bakın" dedi.
- Her satırda önce bütün hash fonksiyonlarının değeri hesaplanır.
- Sonra dokümanlar için bir döngü vardır. Dokümanın o satırdaki girdi değeri 1 ise güncelleme yapılır, değilse bir sonraki dokümana geçilir.
- Güncellemede her hash fonksiyonu için matrisin mevcut değeri ile hash değeri karşılaştırılır ve küçüğü bırakılır.

```
her i, c için: SIG(i, c) = sonsuz
her satır r için:
    her hash fonksiyonu i için h_i(r) hesapla
    her doküman c için:
        eğer M(r, c) == 1:
            her hash fonksiyonu i için:
                SIG(i, c) = min(SIG(i, c), h_i(r))
```

Algoritma bitince elde edilen imza matrisi üzerinden iki doküman arasındaki benzerlik, Jaccard benzerliğine gerek kalmadan, aynı olan satır sayısının toplam satır sayısına oranıyla ölçülür.

> **Ek bilgi (derste anlatılmadı):** Kısa notta geçen genel biçim $h(x) = (ax+b) \bmod n$ büyük olasılıkla kitaptaki öneriye dayanır: a ve b rastgele seçilir, n satır sayısıdır (ya da ondan büyük bir asal sayıdır). Böyle bir fonksiyon satırları bir permütasyona benzer şekilde karıştırır, ama bellekte yalnız a ve b tutulur. Derste kullanılan h_1 ve h_2 bu biçimin özel hâlleridir.

8. Locality Sensitive Hashing (LSH)

8.1 Neden LSH?

Aradan sonra hoca MinHashing'in neden yapıldığını hatırlattı. Büyük doküman kümeleri k uzunluğundaki karakter dizileriyle tanımlanıp 0-1 kolon vektörlerine dönüştürülüyordu. Bu

girdi matrisinde her ikili için Jaccard benzerliği hesaplanıp belirli bir değerin üzerindekiler benzer sayılıyordu. Fakat matris çok büyük olduğundan belleğe sığdırmak zordu. MinHashing ile imza matrisi elde edildi:

- İmza matrisinin kolon sayısı girdi matrisinin kolon sayısıyla aynıdır.
- Azalan, bir dokümanı temsil eden satır sayısıdır. Satır sayısı hash fonksiyonu sayısı kadardır. 100 hash fonksiyonu kullanılırsa imza matrisi 100 satırlıdır.

İmza matrisi küçültülse de hâlâ bütün ikililerin benzerliği hesaplanmalıdır. 1 milyon dokümanda bu 1 milyonun ikili kombinasyonu, yani $\binom{10^6}{2} \approx 5 \times 10^{11}$ ikili demektir. Doküman sayısı 10 milyon, 100 milyon ya da 1 milyar olursa problemin büyüklüğü daha da artar. Bu yüzden şu soru sorulur: Benzerliği bütün ikililer için değil, yalnız yüksek olasılıkla benzer olabilecek ikililer için hesaplamanın bir yolu var mı? Bunu yapan algoritma Locality Sensitive Hashing'dir. İngilizce tanımıyla LSH, benzer olma olasılığı yüksek MinHash imzalarına odaklanır.

8.2 Benzerlik eşiği ve aday ikililer

Algoritma önce bir benzerlik eşiği (similarity threshold) seçilmesini ister. İmza matrisinde iki dokümanın benzerliği (aynı satır sayısı / toplam satır sayısı) bu eşiğin, örneğin %80 ya da %70'in, üzerindeyse bu iki doküman benzer kabul edilir. Algoritmanın amacı, benzer olma olasılığı yüksek aday ikilileri (candidate pairs) bulmak ve gerçek benzerliği yalnız onlar için hesaplamaktır.

8.3 Bantlara bölme

İmza matrisinde kolonlar dokümanların imzaları, satırlar hash fonksiyonlarıdır. LSH imza matrisini b banda (band) böler. Her bantta r satır (row) vardır:

$$n = b \cdot r$$

Hocanın örneği: 100 satırlık imza matrisi $b = 5$ banda bölünürse her bantta $r = 20$ satır olur.

8.4 Bantları kovalara hash'lemek

Bir dokümanın bir banttaki parçası, imza matrisindeki tamsayılardan oluşan küçük bir "çubuk"tur; bu örnekte yukarıdan aşağı 20 tamsayı. Bu 20 tamsayılık dizi bir hash fonksiyonuna verilir. Hash fonksiyonunun alabileceği değerler kovalardır (bucket). Hocanın örneğinde hash fonksiyonu 1 ile 1000 arasında, yani 1000 farklı değer alabilir. Hash fonksiyonu 1 ile 1000 arasında bir değer üretir ve dokümanın adı o kovaya yazılır.

Bu işlem bantdaki bütün dokümanların 20'lik parçaları için yapılır. Sadece bir bant için de değil, bütün bantlar için yapılır. Her dokümanın her banttaki parçası hash'lenir ve doküman adı ilgili kovaya yazılır.

8.5 Aday ikiliden gerçek benzerliğe

İki dokümanın parçaları aynı kovaya düşmüşse bu iki doküman benzer olmaya adaydır. Gerçek benzerlik yalnız aynı kovaya düşen ikililer arasında hesaplanır. Böylece 1 milyonun ikili kombinasyonu yerine yalnız aynı kovaya düşmüş ikililere bakılır. Hoca, iki dokümanın aynı kovaya düşmesinin benzerlik için mutlaka bir işaret olduğunu, aynı kovaya düşmeyenlerin zaten benzer olmadığının düşünüldüğünü söyledi. Örneğin bir kovaya 80 doküman düşmüşse, yalnız bu 80 dokümanın ikili kombinasyonları için benzerlik hesaplanır.

> **Ders sonrası not:** Hoca aday olma koşulunu bir yerde "bantların %80'inden fazlası aynı kovaya düşmüşse" diye anlattı. Standart LSH'de iki dokümanın en az bir bantta aynı kovaya düşmesi aday olmaları için yeterlidir. Derste yapılan hata hesabı da ("20 bandın hiçbirinde aynı olmama") bu standart tanıma dayanır. 0.8 değeri bant oranı değil, benzerlik eşiğidir.

8.6 Yaklaşıklık ve yanlış pozitif

İşin içine bir hash fonksiyonu girdiği anda yaklaşık (approximation) bir sonuçtan söz edilir, çünkü hash fonksiyonunun çözünürlüğü ve başarısı %100 değildir. Normalde birbirine benzemeyen iki doküman yanlışlıkla aynı kovaya eşlenebilir. Aynı kovaya düştükleri için pozitif değerlendirilir ve benzerlikleri hesaplanır, ama bu bir yanlış pozitiftir (false positive). Hoca bunun sebebini hash fonksiyonunun iki farklı dokümanı çözünürlüğü iyi olmadığı için aynı kovaya düşürebilmesi olarak açıkladı.

> **Ek bilgi (derste anlatılmadı):** Kitapta yanlış pozitifin asıl kaynağı büyük olasılıkla kova çakışması değil, benzerliği eşiğin altında olan iki dokümanın bir bantta tesadüfen bütün satırlarda aynı imza değerlerine sahip olmasıdır. Kovalar çok sayıda seçildiğinde, aynı kovaya düşmenin büyük olasılıkla bantların aynı olması anlamına geldiği varsayılır. Yanlış pozitifler, aday ikililerin gerçek benzerliği hesaplanırken elenir. Yanlış negatifler ise (benzer olup hiç aday olmayanlar) bu adımda yakalanamaz.

9. LSH'de Hata Payı: Sayısal Örnek

Hoca "bu yöntemle benzer dokümanları hızlı yakalamak istersem ne gibi bir hatam olabilir?" sorusuyla bir örneğe geçti.

9.1 Verilenler

- İmza matrisinde 100.000 kolon var, yani 100.000 doküman.
- İmza matrisinde 100 satır var, yani 100 MinHash fonksiyonu.
- Her hücre bir tamsayı, 4 bayt.
- Hedef: Benzerliği %80 ($s = 0.8$) olan ikilileri aday olarak bulmak.
- LSH parametreleri: $b = 20$ bant, $r = 5$ satır. Hoca, az önceki örnekte matris 5'e bölünmüştü, şimdi 20'ye bölünüyor ve her banda 5 tamsayı düşüyor dedi ($20 \times 5 = 100$).

9.2 Bellek ve karşılaştırma sayısı

İmza matrisinin bellekte kapladığı yer:

$$100 \times 100.000 \times 4 \text{ bayt} = 40.000.000 \text{ bayt} = 40 \text{ MB}$$

Hocanın vurgusu: İmza matrisinin getirdiği en büyük avantaj az yer kaplamasıdır.

LSH yapılmasaydı 100.000'in ikilisi kadar kolon vektörü karşılaştırılmalıydı:

$$\binom{100.000}{2} = \frac{100.000 \cdot 99.999}{2} = 4.999.950.000 \approx 5 \text{ milyar}$$

Hoca bu kadar ikiliyi karşılaştırmak istemediğimiz için LSH'yi kullandığımızı söyledi.

9.3 Hata payının hesabı

C_1 ve C_2'nin benzerliği 0.8 olsun. Hoca bunu şöyle açıkladı: İmza matrisinin bir satırı için iki dokümanın aynı olma olasılığı 0.8'dir.

Adım 1 (bir bantta aynı olma): Bir bantta 5 satır var. Bandın aynı kovaya düşmesi için 5 satırın hepsinin aynı olması gerekir:

$$0.8^5 = 0.32768 \approx 0.328$$

Adım 2 (bir bantta aynı olmama):

$$1 - 0.328 = 0.672$$

Adım 3 (20 bandın hiçbirinde aynı olmama): Bantlar bağımsız kabul edilirse:

$$(1 - 0.8^5)^{20} = 0.672^{20} \approx 0.00036$$

Hoca bu değerin yaklaşık 3000'de 1 olduğunu söyledi. Python ile hesaplanan değer 0.000356, yani yaklaşık 2800'de 1'dir. Hocanın "3000'de 1" yuvarlaması bununla uyumludur.

Dolayısıyla %80 benzer bir ikilinin aday olarak bulunma olasılığı:

$$1 - (1 - 0.8^5)^{20} \approx 0.99964$$

Hoca bu değeri yanlış negatif (false negative) olarak adlandırdı: Gerçekten %80 benzer bir ikilinin hiçbir bantta aynı kovaya düşmeyip gözden kaçma olasılığı yaklaşık 3000'de 1'dir.

> **Ders sonrası not:** Altyazıda değer "0.3035" diye geçiyor. Doğru hesap $(1 - 0.8^5)^{20} = 0.672^{20} \approx 0.00036$'dır. Bu değer hocanın söylediği "3000'de 1" ile uyumludur. Değer büyük olasılıkla slaytta "0.00035" olarak yazıyordu ve altyazıya yanlış geçti.
>
> **Ders sonrası not:** Hoca değeri doğru olarak "yanlış negatif" diye adlandırdı, ama hemen ardından "hash fonksiyonunun bu 80 dokümanın içine orada olmaması gereken dokümanları koymuş olabilmesi", "benzerliği %80'in altında olan ikilileri kovaya koyabilme olasılığı" diye açıkladı. Bu açıklama yanlıştır. Benzer olmayan dokümanların aynı kovaya konması yanlış pozitiftir. 0.00036 değeri, gerçekten %80 benzer bir ikilinin hiçbir bantta aynı kovaya düşmeme, yani kaçırılma olasılığıdır.
>
> **Ek bilgi (derste anlatılmadı):** Genel olarak benzerliği s olan bir ikilinin b bant, r satırla aday olma olasılığı $1 - (1 - s^r)^b$'dir. s'ye göre çizildiğinde bu fonksiyon S biçimli bir eğridir. Eğrinin dik yükseldiği eşik büyük olasılıkla yaklaşık $(1/b)^{1/r}$ civarındadır. b ve r seçimi, yanlış pozitif ile yanlış negatif arasındaki dengeyi ayarlar.

9.4 Yöntemin özeti

Hoca iki yolu karşılaştırarak bitirdi:

- **Elle (manuel) yöntem:** İmza matrisindeki dokümanların bütün ikili kombinasyonlarının benzerliği bulunur, eşiğin üzerindekiler benzer kabul edilir.
- **LSH yöntemi:** İmza matrisindeki dokümanların bantlarındaki parçaları bir hash fonksiyonuyla kovalara eşlenir. Aynı kovaya düşenler aday ikili olarak alınır. Benzerlik yalnız aday ikililer için hesaplanır. Örneğin 100.000 doküman içinde aynı kovaya 100 doküman düşmüşse yalnız bu 100'ün ikilileri için benzerlik değerlerine bakılır.

Böylece kontrol edilmesi gereken ikililerin sayısı azalır. Bedeli, küçük bir olasılıkla hata yapılmasıdır.

10. Kapanış: Örnek Sorular ve Sonraki Ders

Hoca LSH ile ilgili bir ve Shingling ile ilgili bir örnek soruyu, ayrıca daha önce işlenen MapReduce ile ilgili bir örneği cevaplarıyla birlikte dersin sayfasına koyacağını söyledi. Bu derste 3. bölüm tamamlandı. Gelecek hafta 4. bölüme sıfırdan başlanacak. Ayrıca dönem projesinin MapReduce programlama ile büyük veri işleme ve analiz algoritmaları kullanılarak yapılacak bir uygulama olacağı söylendi.

Örnek Sorular

Bu derste hazır kare ve video yoktur. Aşağıdaki sorular hocanın derste sözlü olarak sorup cevapladığı sorulardır.

Soru 1 (derste sözlü, 00:20:30): Doküman `abcab` ve $k = 2$ ise shingle kümesi nedir?

> **Çözüm:** $\{ab, bc, ca\}$. `ab` iki kez geçse de küme olduğu için bir kez yazılır.

Soru 2 (derste sözlü, 00:36:24-00:37:07): Slayttaki 3 satırlı, 4 kolonlu imza matrisinde C_1-C_3, C_2-C_4 ve C_1-C_2 ikililerinin benzerliği nedir?

> **Çözüm:** Aynı satır sayısı / toplam satır sayısı. C_1-C_3: $2/3$. C_2-C_4: $3/3 = 1$. C_1-C_2: $0/3 = 0$.

Soru 3 (derste sözlü, 00:41:19): Permütasyon tabanlı MinHash'te 2. hash fonksiyonu için 3. dokümanın imza değeri nedir? Permütasyon sırasında 1., 2., 3. satırlarda 0, 4. satırda 1 vardır.

> **Çözüm:** İlk 1 görülen sıra 4 olduğu için değer 4'tür.

Soru 4 (derste sözlü, 00:43:46-00:50:32): 5 satırlı girdi matrisinde $C_1 = (1, 0, 1, 1, 0)$, $C_2 = (0, 1, 1, 0, 1)$ ve hash fonksiyonları $h_1(x) = x \bmod 5$, $h_2(x) = (2x + 1) \bmod 5$ ise imza matrisi nedir?

> **Çözüm:** Bölüm 7.3'teki adımlarla C_1'in imzası $(1, 2)$, C_2'nin imzası $(0, 0)$ olur. Bu imzalarda iki satır da farklıdır, imza benzerliği $0/2 = 0$'dır (ara adım eklendi; hoca bu ikili için benzerliği söylemedi).

Soru 5 (derste sözlü, 01:08:30): 100 hash fonksiyonu kullanılırsa imza matrisinin satır sayısı ne olur?

> **Çözüm:** Satır sayısı hash fonksiyonu sayısı kadardır: 100. Kolon sayısı girdi matrisiyle aynı kalır.

Soru 6 (derste sözlü, 01:22:42-01:28:13): 100.000 doküman, 100 MinHash fonksiyonu, $b = 20$, $r = 5$ ve benzerlik 0.8 iken imza matrisinin bellek boyutu, LSH olmadan karşılaştırılacak ikili sayısı ve %80 benzer bir ikilinin hiçbir bantta aday olmama olasılığı nedir?

> **Çözüm:** Bellek: $100 \times 100.000 \times 4 = 40$ MB. İkili sayısı: $\binom{100.000}{2} \approx 5$ milyar. Bir bantta aynı olma: $0.8^5 \approx 0.328$. Bir bantta aynı olmama: 0.672. Hiçbir bantta aynı olmama: $0.672^{20} \approx 0.00036$, yaklaşık 3000'de 1 (yanlış negatif).

Dersin Toparlaması

Bu derste 3. bölümün üç aşamalı süreci tamamlandı. Önce dokümanlar shingling ile k uzunluğundaki karakter dizilerinden oluşan kümelere çevrildi ve bu kümeler 0-1 kolon vektörlerinden oluşan bir girdi matrisine dönüştürüldü. İki doküman arasındaki benzerlik Jaccard benzerliğiyle ölçülüyordu. MinHashing ile bu büyük matris, satır sayısı hash fonksiyonu sayısına inen küçük bir imza matrisine çevrildi. İmza matrisindeki "aynı satır oranı", Jaccard benzerliğine yakınsar. Permütasyon tabanlı hash fonksiyonları bellek gerektirdiği için, satır numarasını alan basit hash fonksiyonları ve satır satır minimum güncelleyen bir algoritma kullanıldı.

Son aşama LSH, imza matrisinde bile çok fazla olan ikili karşılaştırma sayısını azaltır. İmza matrisi b banda bölünür, her banttaki parçalar kovalara hash'lenir ve yalnız aynı kovaya düşen aday ikililerin benzerliği hesaplanır. Yöntem yaklaşıktır: Sayısal örnekte %80 benzer bir ikilinin kaçırılma olasılığı yaklaşık 3000'de 1 çıktı. Gelecek derste 4. bölüme geçilecek.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.